Introduction

Namespace F23.StringSimilarity

Classes

Cosine

Damerau

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

Jaccard

Each input string is converted into a set of n-grams, the Jaccard index is then computed as |V1 inter V2| / |V1 union V2|. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as 1 - cosine similarity. Jaccard index is a metric distance.

JaroWinkler

The Jaro–Winkler distance metric is designed and best suited for short strings such as person names, and to detect typos; it is (roughly) a variation of Damerau-Levenshtein, where the substitution of 2 close characters is considered less important then the substitution of 2 characters that a far from each other. Jaro-Winkler was developed in the area of record linkage (duplicate detection) (Winkler, 1990). It returns a value in the interval [0.0, 1.0]. The distance is computed as 1 - Jaro-Winkler similarity.

Levenshtein

The Levenshtein distance between two words is the Minimum number of single-character edits (insertions, deletions or substitutions) required to change one string into the other.

LongestCommonSubsequence

The longest common subsequence (LCS) problem consists in finding the longest subsequence common to two (or more) sequences. It differs from problems of finding common substrings: unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences.

It is used by the diff utility, by Git for reconciling multiple changes, etc.

The LCS distance between Strings X (length n) and Y (length m) is n + m - 2 |LCS(X, Y)| min = 0 max = n + m

LCS distance is equivalent to Levenshtein distance, when only insertion and deletion is allowed (no substitution), or when the cost of the substitution is the double of the cost of an insertion or deletion.

! This class currently implements the dynamic programming approach, which has a space requirement O(m * n)!

MetricLCS

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkelund.

NGram

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

NormalizedLevenshtein

This distance is computed as levenshtein distance divided by the length of the longest string. The resulting value is always in the interval [0.0 1.0] but it is not a metric anymore! The similarity is computed as 1 - normalized distance.

OptimalStringAlignment

Provides an implementation of the Optimal String Alignment (OSA) distance algorithm, which calculates the minimum number of operations required to transform one string into another. Supported operations include insertion, deletion, substitution of a single character, and transposition of two adjacent characters, with the constraint that no substring is edited more than once.

QGram

Q-gram distance, as defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches". The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurences of each n-gram): SUM(|V1_i - V2_i|). Q-gram distance is a lower bound on Levenshtein distance, but can be computed in O(m + n), where Levenshtein requires O(m.n).

RatcliffObershelp

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings a the doubled number of matching characters divided by the total number of characters in the two strings.Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as 1 - Ratcliff/Obershelp similarity.

Author: Ligi https://github.com/dxpux (as a patch for fuzzystring) Ported to java from .net by denmase Ported back to .NET by paulirwin to retain compatibility with upstream Java project

ShingleBased

Base class for shingle based algorithms.

SorensenDice

Similar to Jaccard index, but this time the similarity is computed as 2 * |V1 inter V2| / (|V1| + |V2|). Distance is computed as 1 - cosine similarity.

WeightedLevenshtein

Implementation of Levenshtein that allows to define different weights for different character substitutions.

Interfaces

ICharacterInsDel

As an adjunct to ICharacterSubstitution, this interface allows you to specify the cost of deletion or insertion of a character.

ICharacterSubstitution

Used to indicate the cost of character substitution.

Cost should always be in [0.0 .. 1.0] For example, in an OCR application, cost('o', 'a') could be 0.4 In a checkspelling application, cost('u', 'i') could be 0.4 because these are next to each other on the keyboard...